LIFETRACK HEALTH

A Patient-Owned, Local-First Health Record with Consent-Controlled Sharing

NTCC Project Report

submitted in partial fulfilment of the requirements for the degree of

Bachelor of Science in Information Technology (B.Sc. IT)

Submitted by

Anish Raj

Enrolment No. A45304925031

Under the supervision of

Dr. Priyadarshini Gupta

Assistant Professor, Amity Institute of Information Technology

AMITY INSTITUTE OF INFORMATION TECHNOLOGY

AMITY UNIVERSITY PATNA

Patna – 801503

Academic Session 2025–2028 · October 2026

DECLARATION

I hereby declare that the project report entitled **"LifeTrack Health: A Patient-Owned, Local-First Health Record with Consent-Controlled Sharing"**, submitted to Amity Institute of Information Technology, Amity University Patna, in partial fulfilment of the requirements for the degree of Bachelor of Science in Information Technology (B.Sc. IT), is a record of original work carried out by me under the supervision of Dr. Priyadarshini Gupta.

The software described in this report was designed, built and tested by me. Published standards, laws and libraries used in the work are acknowledged in the References, and the software tools used, including an AI coding assistant, in the Acknowledgement. This report has not been submitted, in part or in full, to any other university or institution for the award of any degree or diploma.

Anish Raj

Enrolment No. A45304925031

Signature: _____

Place: Patna Date: _____

CERTIFICATE

This is to certify that the project report entitled **“LifeTrack Health: A Patient-Owned, Local-First Health Record with Consent-Controlled Sharing”**, submitted by Anish Raj (Enrolment No. A45304925031) in partial fulfilment of the requirements for the degree of Bachelor of Science in Information Technology (B.Sc. IT) at Amity Institute of Information Technology, Amity University Patna, is a bona fide record of work carried out under my supervision during the academic session 2025–2028.

To the best of my knowledge, the work presented in this report is original and has not been submitted elsewhere for the award of any degree or diploma.

Dr. Priyadarshini Gupta

Assistant Professor

Amity Institute of Information Technology, Amity University Patna

Signature: _____ Date: _____

ACKNOWLEDGEMENT

I express my sincere gratitude to my supervisor, **Dr. Priyadarshini Gupta**, for her guidance, encouragement and critical feedback throughout this project. Her questions about how an analytical or clinical claim can be proven correct shaped the testing approach described in this report.

I thank the Director and faculty of Amity Institute of Information Technology, Amity University Patna, for providing the environment and resources to carry out this work, and my classmates for testing early versions of the apps and reporting what confused them.

I also acknowledge the open-source communities behind Flutter, Dart, the `cryptography` package, Tesseract OCR and the many other libraries this project builds on.

Parts of the code and of this report were drafted with the help of an AI coding assistant (Claude, by Anthropic). I directed the work, reviewed what it produced, and ran and checked the tests reported here.

Finally, I thank my family for their patience and support.

Anish Raj

ABSTRACT

In India a patient's medical history is usually spread across the clinics, hospitals and laboratories they have visited, with the patient carrying paper reports between them. When the history is not available, investigations are repeated, medicines are duplicated and family patterns are missed. The cause is usually scattered data, not poor clinical judgement. The patient is the only person present at every encounter, yet rarely holds a complete record.

LifeTrack Health is a working prototype of a health record that the patient owns and keeps on their own phone. It has four parts: an Android patient app built in Flutter; LifeTrack Clinic, a doctor app; a small Dart server that acts as a doctor registry, an encrypted relay and a backup store; and a shared protocol package. Records live on the patient's phone, sealed with AES-256-GCM under a hardware-backed device key. A doctor sees records only after the patient scans the doctor's registry-verified code and approves a level of access (Summary, All reports, Family illnesses or Home readings) for a chosen time between one hour and two days. The share is encrypted on the phone to that doctor's device key alone (X25519 with AES-GCM); the server carries it but can never open it. Prescriptions return to the patient signed with the doctor's Ed25519 key.

The patient app also reads photographed or PDF lab reports on the device with Tesseract OCR, compares results with published reference ranges without ever stating a diagnosis, tracks medicines with reminders, keeps a child vaccination schedule, manages a family under a head-of-family model, offers an Emergency ID readable from the lock screen, and makes an end-to-end encrypted backup protected by an app-generated password and Shamir 2-of-3 recovery codes. Every read, share, refusal and export is written to an append-only audit log before it happens.

The system is written in Dart and Flutter (about 21,700 lines of application code). It has 354 automated tests at unit, widget, invariant and end-to-end integration level, and all of them pass with a clean static analysis. The invariant tests turn the project's privacy promises into checks that fail the build: no screen may claim the server "cannot read" the data, for example, and no agent may issue itself a grant. The principal open items are ABDM/ABHA integration and a legal review before any pilot with real patients.

Keywords: personal health record, patient consent, end-to-end encryption, local-first software, Flutter, digital health India, DPDP Act 2023.

TABLE OF CONTENTS

Chapter 1: Introduction **1**
1.1 Background 1
1.2 Problem statement 1
1.3 Motivation 2
1.4 Objectives 2
1.5 Scope 2
1.6 Methodology 3
1.7 Organisation of the report 3
Chapter 2: Literature Review and Existing Systems **4**
2.1 Categories of existing solutions 4
2.2 Relevant standards 4
2.3 Legal and regulatory context 5
2.4 Design principles drawn from the literature 5
2.5 Research gap 5
Chapter 3: Requirements Analysis **7**
3.1 Stakeholders 7
3.2 Functional requirements 7
3.3 Non-functional requirements 8
3.4 Hardware and software requirements 9
3.5 Use cases 9
3.6 Feasibility study 10
Chapter 4: System Design **11**
4.1 Overall architecture 11
4.2 Data flow (Level 0) 11
4.3 Patient app architecture 12
4.4 Consent model 13
4.5 Data design 14
4.6 Security design 14
4.7 LifeTrack ID 16
4.8 Entity-relationship model 17
4.9 Key algorithms 17
Chapter 5: Implementation **19**
5.1 Technology stack 19
5.2 Code organisation and size 19
5.3 Patient app 19
5.4 Doctor app: LifeTrack Clinic 22
5.5 Server 24
5.6 Development practices 24
5.7 Selected implementation details 24
Chapter 6: Testing and Results **27**
6.1 Testing strategy 27
6.2 Results 27
6.3 Selected test cases 28

6.4 Manual testing on a device....28
6.5 Example of an invariant test....29
Chapter 7: Results and Discussion....30
7.1 Objectives achieved....30
7.2 Changes from the original proposal....30
7.3 Discussion....31
7.4 Comparison with existing solutions....31
Chapter 8: Conclusion and Future Scope....32
8.1 Conclusion....32
8.2 Limitations....32
8.3 Future scope....32
References....34
Appendix A: Version History....36
Appendix B: Running the System....37

LIST OF FIGURES

Figure 3.1 Use-case diagram
Figure 4.1 System architecture
Figure 4.2 Level 0 data flow diagram
Figure 4.3 Layers of the patient app
Figure 4.4 Clinical share and prescription sequence
Figure 4.5 Backup key hierarchy
Figure 4.6 Entity-relationship diagram
Figure 5.1 Patient app: Reminders, Records and Insights
Figure 5.2 Patient app: Share and Me
Figure 5.3 LifeTrack Clinic: verified code, request by ID and an arrived share
Figure 5.4 LifeTrack Clinic: a shared summary and a prescription being written
Figure 5.5 LifeTrack Clinic: the Profile tab

LIST OF TABLES

Table 2.1 Existing solution categories compared
Table 3.1 Stakeholders and their needs
Table 3.2 Functional requirements: patient app
Table 3.3 Functional requirements: doctor app and server
Table 3.4 Non-functional requirements
Table 3.5 Hardware and software requirements
Table 3.6 Main use cases
Table 4.1 Access levels
Table 4.2 Trend thresholds used by Plane A
Table 5.1 Technology stack
Table 5.2 Code size by part
Table 5.3 Server routes
Table 6.1 Test results by part
Table 6.2 Selected test cases
Table 7.1 Objectives and outcomes
Table 7.2 Comparison with existing categories
Table A.1 Version history

LIST OF ABBREVIATIONS

ABDM	Ayushman Bharat Digital Mission
ABHA	Ayushman Bharat Health Account
AES-GCM	Advanced Encryption Standard, Galois/Counter Mode
APK	Android Package
DFD	Data Flow Diagram
DPDP	Digital Personal Data Protection (Act, 2023)
EMR	Electronic Medical Record
ER	Entity-Relationship
GF	Galois Field
FHIR	Fast Healthcare Interoperability Resources
HKDF	HMAC-based Key Derivation Function
OCR	Optical Character Recognition
QR	Quick Response (code)
SaMD	Software as a Medical Device
UI	User Interface
UIP	Universal Immunisation Programme
WCAG	Web Content Accessibility Guidelines

Chapter 1: Introduction

1.1 Background

In India, health care comes from a very large number of independent providers: government hospitals, private hospitals, single-doctor clinics, diagnostic laboratories and pharmacies. Each keeps its own records, if it keeps them digitally at all, and very few exchange them. For the patient, the practical result is a carry bag of paper (prescriptions, lab printouts, discharge summaries and scan reports) that is incomplete, easily lost and hard for a new doctor to read in a ten-minute consultation.

The Government of India's Ayushman Bharat Digital Mission (ABDM) is building national rails for health data exchange, including the ABHA health ID and consent managers. At the same time, the Digital Personal Data Protection Act, 2023 (DPDP Act) makes consent, purpose limitation and the right to access one's data legal obligations. Both point in the same direction: the patient should be at the centre of their own record and should decide who sees it.

1.2 Problem statement

A patient is the only person present at every one of their medical encounters, but has no single, trustworthy, consolidated view of their own history and no safe way to show the relevant part of it to a new doctor. Three consequences follow:

- **Repeated investigations.** When a previous test result is not available, or its source cannot be trusted, the test is ordered again, costing the patient money and time.
- **Duplicate or conflicting medication.** A doctor who does not know what the patient already takes may prescribe the same ingredient under a different brand name.
- **Missed hereditary patterns.** Illnesses in the family are rarely recorded in a way a doctor can use.

Existing solutions either keep the data inside one institution (hospital EMRs), are designed for large organisations (enterprise systems), establish identity without holding the record (government health IDs), or store documents without making them useful or safely shareable (document-locker apps). The project therefore asks: **can a patient keep a complete, useful health record on their own phone and share exactly the right part with exactly the right doctor, for a limited time, without any server ever being able to read it?**

1.3 Motivation

Health data is among the most sensitive personal data. A design in which a company's servers hold readable copies of millions of records creates a single point of failure for breaches, misuse and function creep. In a local-first design the main copy lives on the patient's device and leaves it only encrypted, and only when the patient asks. That removes the central store of readable records. Building such a system end to end, with the cryptography, the consent rules and a doctor-side app, requires work across mobile development, security, distributed systems, user-interface design and law, which made it a suitable scope for this project.

1.4 Objectives

1. Build a patient-owned record on the phone that holds prescriptions, lab reports, scans, notes, allergies, conditions, vitals and vaccinations for every member of a family, with all data encrypted at rest.
2. Make clinician access explicit, time-limited and revocable: the patient approves a level of access for one verified doctor for a chosen time, and can shorten, extend or end it.
3. Make sure no server can read shared records or backups: shares are encrypted to the doctor's device key and backups to a key derived from a password only the patient holds.
4. Provide useful, safe insights on the device: compare results with published reference ranges, notice movement between reports and spot duplicate medicines, without ever stating a diagnosis or a forecast.
5. Reduce typing by reading photographed and PDF lab reports on the device (OCR), with the patient confirming uncertain fields.
6. Build a doctor app with a verified identity, so that a patient knows who they are sharing with, and so that prescriptions returned to the patient are signed.
7. Make every privacy promise verifiable: express the system's rules as automated tests that fail the build when broken.

1.5 Scope

In scope. An Android patient app; a doctor app (Flutter, runs in a browser or on a phone); a local server providing a doctor registry, an encrypted relay, a LifeTrack ID directory, a backup vault and an identity-check flow; and a shared protocol package. Sample (synthetic) patients and doctors are used throughout.

Out of scope for this phase. Real patient data; live ABDM/ABHA exchange; real DigiLocker partner integration (a “practice” stand-in drives the same flow); cloud-based AI analysis (designed as “Plane C” but disabled); iOS OCR; clinical decision support and predictive modelling; deployment on the public internet.

1.6 Methodology

Development followed an incremental model in five stages. Each stage ended with working, tested software rather than documents alone.

1. Requirements and threat analysis: the patient, doctor and family needs listed in Chapter 3, together with the ways health data can leak (a readable server copy, an over-broad grant, a misleading claim on screen).
2. Architecture and invariants: the three-plane structure, the single consent authority and eighteen written invariants (rules the code must never break), each tied to a test.
3. Incremental feature releases: each feature (records, insights, sharing, reminders, family, backup, OCR and so on) was built as a separate version with its own tests and a changelog line.
4. Security review rounds: two dedicated hardening releases fixed issues found by reviewing the code against the invariants.
5. Integration and device testing: both apps were run against the real server handler in automated integration tests, and the release build was tested by hand on an Android phone.

The guiding rule across all stages, taken from the project proposal, was that a claim about the system is accepted only when a test can prove it.

1.7 Organisation of the report

Chapter 2 reviews existing systems, standards and laws. Chapter 3 sets out the requirements and feasibility. Chapter 4 describes the architecture, data design and security design. Chapter 5 explains the implementation, module by module, with screenshots. Chapter 6 covers testing and results. Chapter 7 discusses outcomes against the objectives, and Chapter 8 concludes with limitations and future work. Appendices give the version history and instructions to run the system.

Chapter 2: Literature Review and Existing Systems

2.1 Categories of existing solutions

Table 2.1: Existing solution categories compared

Category	Examples	Strengths	Gaps for the patient
Hospital EMR / HIS	Systems used inside a hospital chain	Complete within the institution; integrated with billing and labs	Data stays inside the institution; patient has no consolidated view
Enterprise health platforms	Large vendor suites	Powerful, standards-based	Expensive and complex; built for organisations, not individuals
Government health ID and exchange	ABHA, ABDM consent managers, PHR apps	National identity and consent rails; legal backing	Identity and exchange rather than a patient-held, analysed record; depends on provider adoption
Document lockers	DigiLocker and general cloud storage	Easy storage of issued documents	Stores files, does not interpret results; provider holds readable copies
Personal record apps	Consumer health-record apps	Convenient capture and reminders	Usually cloud-first with readable server copies; sharing is often an open link

2.2 Relevant standards

- **HL7 FHIR R4** [8]. HL7's interoperability standard models clinical data as typed resources such as Patient, Observation, MedicationRequest and Provenance. ABDM uses FHIR-based profiles. LifeTrack's record model keeps provenance and source on every record so that a later FHIR export can say where each value came from; FHIR export itself is future work (Section 8.3).
- **Ayushman Bharat Digital Mission (ABDM)** [6]. Defines the ABHA number, health information providers and users, and consent artefacts. LifeTrack records an ABHA number as optional, typed in, and shown as "not verified" until ABDM integration exists.
- **Cryptographic standards.** AES-GCM (NIST SP 800-38D) [9] for authenticated encryption; X25519 (RFC 7748) [10] for key agreement; Ed25519 (RFC 8032) [11] for signatures; HKDF (RFC 5869) [12] for key derivation; Argon2id (RFC 9106) [13] for password hashing, using the OWASP-recommended minimum parameters [14]; Shamir's secret sharing [15] for recovery codes.
- **WCAG 2.1 AA** [19]. Contrast requirements for readable text; the apps are tested against them in both light and dark mode.

2.3 Legal and regulatory context

- **Digital Personal Data Protection Act, 2023** [1]. Under the Act, a person must agree knowingly and voluntarily to each use of their data, data may be used only for the purpose agreed, people may see and erase what is held about them, and a parent must consent for a child (Section 9). LifeTrack implements consent per purpose and time, an export of all data (the access right), erasure, and parent-declared consent for children that stops at 18.
- **Aadhaar Act, 2016, Section 29** [2] restricts storing and sharing Aadhaar numbers. LifeTrack never asks for or stores an Aadhaar number; LifeTrack IDs always begin with 1 so that they can never be, or validate as, an Aadhaar number.
- **Justice K.S. Puttaswamy v. Union of India (2018)** [3] and the 2019 Aadhaar amendment [2] limit mandatory Aadhaar authentication by private entities. The project's optional "identity required" mode, which verifies every profile through DigiLocker, is therefore flagged for legal review before any pilot (Section 8.2).
- **Telemedicine Practice Guidelines, 2020** [5] require a doctor to keep a record of the consultation and prescription. LifeTrack therefore never promises that a doctor "keeps no copy"; it says that shared records close when access ends.
- **Information Technology (SPDI) Rules, 2011** [4] apply to sensitive personal data, including health data, until the DPDP Rules commence.

2.4 Design principles drawn from the literature

Kleppmann et al. [17] describe local-first software, in which the main copy of the data stays on the user's own device and the network is used only to sync it and work with others. End-to-end encryption, familiar from messaging, shows that a relay can carry data it cannot read. Clinical-safety guidance for software as a medical device [21] distinguishes between displaying information (low risk) and diagnosing or predicting (regulated). LifeTrack combines these: a local-first record, an end-to-end encrypted relay, and insights that state only position and movement and route the patient to a person.

2.5 Research gap

A review of available systems found none that combines (a) a patient-held, encrypted, family-aware record, (b) on-device interpretation of results that is deliberately non-diagnostic, (c) sharing to a registry-verified doctor that the server cannot read, with a time limit the patient

sets, and (d) privacy promises enforced by automated tests. LifeTrack Health is a prototype of that combination.

Chapter 3: Requirements Analysis

3.1 Stakeholders

Table 3.1: Stakeholders and their needs

Stakeholder	Needs
Patient / head of family	Keep everyone’s records in one place; understand results; share safely with a doctor; get reminders; recover the record if the phone is lost
Doctor	See a trustworthy summary quickly, with where each result came from; ask a patient for access remotely; send a prescription back
Registry administrator	Approve, reject and revoke doctors; keep the registry key safe
Linked family member	See a parent’s Emergency ID and current medicines when the parent chooses to share them
Paramedic / bystander	Read the Emergency ID from the lock screen in an emergency

3.2 Functional requirements

Patient app

Table 3.2: Functional requirements: patient app

ID	Requirement
FR-1	Create profiles for the phone owner (head of family) and family members with roles; children only as the head’s son or daughter
FR-2	Add, view, edit and delete prescriptions, reports, scans and notes; keep the original image or PDF with each record
FR-3	Read a photographed or PDF lab report on the device and turn it into a draft record; uncertain fields must be confirmed by the patient
FR-4	Show each test’s results over time; compare with published reference ranges for age and sex; mark unit mismatches as “not checked”
FR-5	Detect duplicate medicines (same ingredient, different brand) and meaningful changes between reports
FR-6	Medicine reminders from active prescriptions, plus the patient’s own reminders; names hidden on the lock screen by default
FR-7	Child vaccination schedule (India Universal Immunisation Programme) with due dates from date of birth
FR-8	Home readings diary for blood pressure and sugar, with charts
FR-9	Emergency ID (blood type, conditions, medicines, contact) readable from the lock screen, every read logged; plain-text QR
FR-10	Share with a doctor by scanning the doctor’s verified code; choose level and time (1 h, 4 h, 12 h, 1 day, 2 days); change or end it later
FR-11	Receive and approve or decline access requests a doctor sends by LifeTrack ID
FR-12	Collect signed prescriptions from the doctor and file them as doctor-issued
FR-13	Encrypted backup of all profiles to the server; restore onto an empty phone with the password or recovery codes
FR-14	Export all data as a zip (HTML, JSON, originals, access log), password-protected by default

ID	Requirement
FR-15	Access log showing every read, share, refusal, export and Emergency ID open
FR-16	Family link: share Emergency ID and/or medicine list with a linked adult's phone, each off by default

Doctor app and server

Table 3.3: Functional requirements: doctor app and server

ID	Requirement
FR-17	Doctor enrols with name, registration number and specialty; the device generates its own keys; the administrator approves
FR-18	Doctor shows a verified QR code for the patient to scan; certificate lasts 30 days and renews automatically
FR-19	Doctor requests access by a patient's 12-digit LifeTrack ID, asking for levels and a time
FR-20	Doctor sees shared records with provenance and a countdown; access closes at the end time
FR-21	Doctor writes, signs, prints and sends a prescription back to the patient
FR-22	Doctor keeps their own sealed notes and a patient register
FR-23	Server: registry with admin approve/reject/revoke; relay that deletes on pickup or expiry; ID directory; backup vault with rate-limited proofs; identity-check flow

3.3 Non-functional requirements

Table 3.4: Non-functional requirements

Quality	Requirement and how it is met
Confidentiality	Every blob on disk sealed with AES-256-GCM; shares and backups end-to-end encrypted; server holds no key that opens records
Integrity	Authenticated encryption binds each blob to its name; prescriptions and certificates are signed; audit log is append-only
Accountability	Every read, change, grant, refusal, egress, restore and erasure is audited; egress is logged before it happens
Availability / offline	The patient app works fully offline; the network is needed only to share, back up, or check a doctor's status
Usability	Plain-language screens; one confirmation for scanned reports; levels as presets; dark mode
Accessibility	All visible text meets WCAG AA contrast in light and dark mode (tested); semantic colours checked for colour-vision deficiency
Safety	Insights never diagnose or forecast; banned phrases are dropped, not rephrased; adult ranges never applied to children or in pregnancy
Maintainability	Strict static analysis; 354 automated tests; every update raises the version and adds a changelog line
Portability	Flutter: one code base for the patient app (Android) and the doctor app (web or mobile)

3.4 Hardware and software requirements

Table 3.5: Hardware and software requirements

Item	Development	Deployment (prototype)
Hardware	Laptop with 8 GB+ RAM; Android phone with USB debugging	Android 8.0+ phone with a screen lock; any modern browser for the doctor app
Operating system	macOS / Windows / Linux	Android; server runs on any OS with Dart
Languages	Dart 3, Kotlin (Android platform code)	Same
Frameworks	Flutter, shelf (server)	Same
Tools	Android SDK, adb, Git and GitHub, VS Code / Android Studio	Not needed
Key libraries	cryptography, flutter_secure_storage, local_auth, flutter_zxing, qr_flutter, Tesseract4Android, flutter_local_notifications, archive, printing	Bundled in the apps

3.5 Use cases

Figure 3.1 shows the actors and the main use cases. The patient (as head of the family) has the most; a doctor reaches the system only through verified enrolment and patient-approved shares.

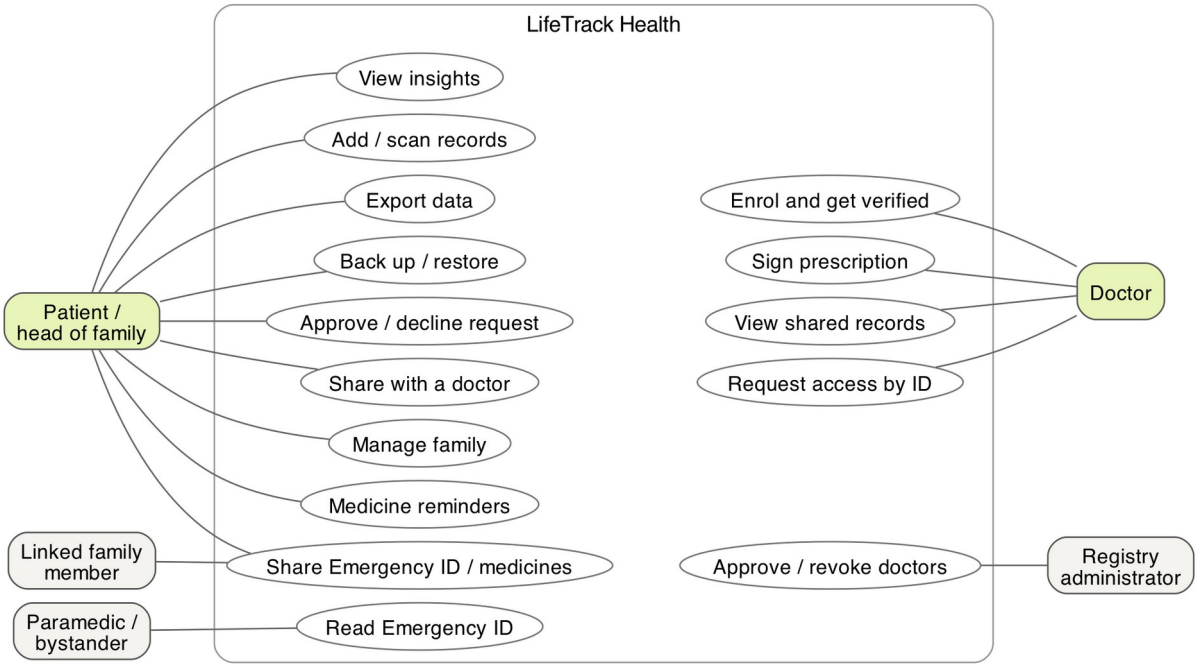


Figure 3.1: Use-case diagram

Table 3.6: Main use cases

ID	Use case	Actor	Main flow
UC-1	Add or scan a record	Patient	Types a record, or photographs/imports a report; OCR fills a draft; the patient ticks uncertain fields and confirms.

ID	Use case	Actor	Main flow
UC-2	Share with a doctor	Patient	Scans the doctor's code; the phone verifies the certificate; the patient picks a level and time; the encrypted bundle is sent.
UC-3	Respond to an access request	Patient	A signed request from a verified doctor appears; the patient approves (possibly at a lower level or shorter time) or declines.
UC-4	End or change access	Patient	From "Who has access now", shortens, extends or ends a share; the doctor's copy is replaced or closed.
UC-5	Back up and restore	Patient	Turns on backup and receives the generated password and recovery codes; on a new, empty phone restores with them.
UC-6	Read the Emergency ID	Paramedic	Opens the card from the lock screen without unlocking; the read is logged.
UC-7	Enrol and get verified	Doctor	Enters registration details; the administrator approves; the device receives a signed 30-day certificate.
UC-8	View records and prescribe	Doctor	Opens an arrived share, reviews the summary, writes and signs a prescription that returns to the patient.
UC-9	Approve or revoke a doctor	Administrator	Reviews pending enrolments and approves, rejects or revokes them with the admin token.

3.6 Feasibility study

- **Technical.** Flutter and Dart [20] provide mature cryptography, camera, QR and notification support; Tesseract runs fully on-device. Every required flow works end to end in the prototype.
- **Economic.** All tools and libraries are free and open source. The server is a single small Dart process; hosting costs would be minimal because it stores only ciphertext and short-lived messages.
- **Operational.** Patients already use smartphones and QR codes; doctors already use computers in clinics. The share flow requires one scan and one approval.
- **Legal.** The design follows DPDP principles and avoids storing Aadhaar numbers. Mandatory identity verification and the clinical status of insights need confirmation from legal counsel before a pilot.

Chapter 4: System Design

4.1 Overall architecture

LifeTrack Health has four parts (Figure 4.1). The patient app holds the record. The doctor app, LifeTrack Clinic, is the only other place records can be read, and only while a share is open. The server is kept small and is never trusted with readable content: it signs doctor certificates, carries sealed messages, maps LifeTrack IDs to mailboxes, stores encrypted backups and runs the identity-check flow. A shared Dart package, lifetrack_protocol, defines the message formats, certificates and ID rules used by all three, so that the apps and the server cannot drift apart.

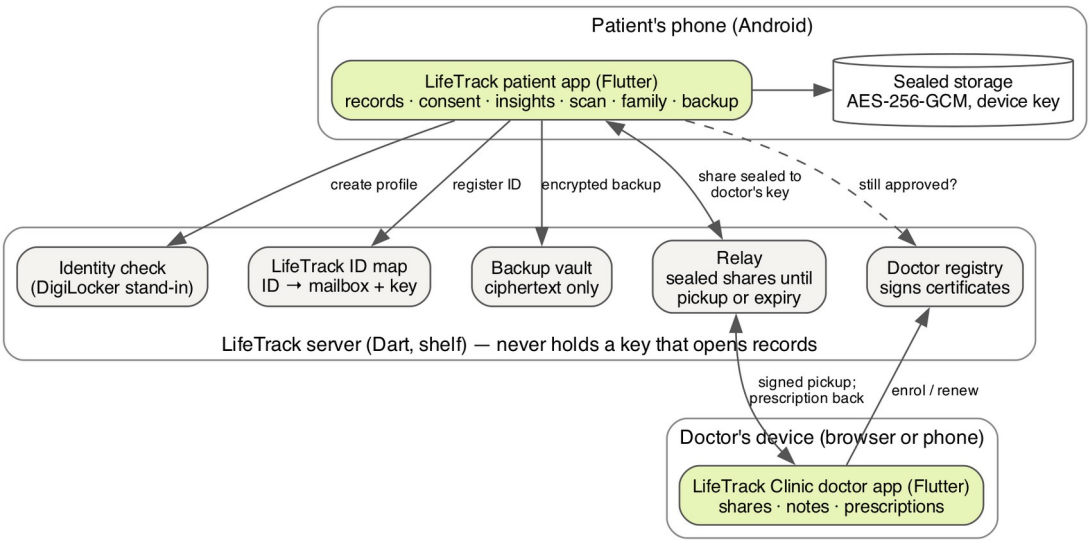


Figure 4.1: System architecture

4.2 Data flow (Level 0)

Figure 4.2 shows the system as a single process with its external entities. Records enter only from the patient (typed, scanned, imported) or from a doctor (signed prescriptions). Records leave only in three ways: a clinical share the patient approved, encrypted backup ciphertext, or a family-link message the patient ticked.

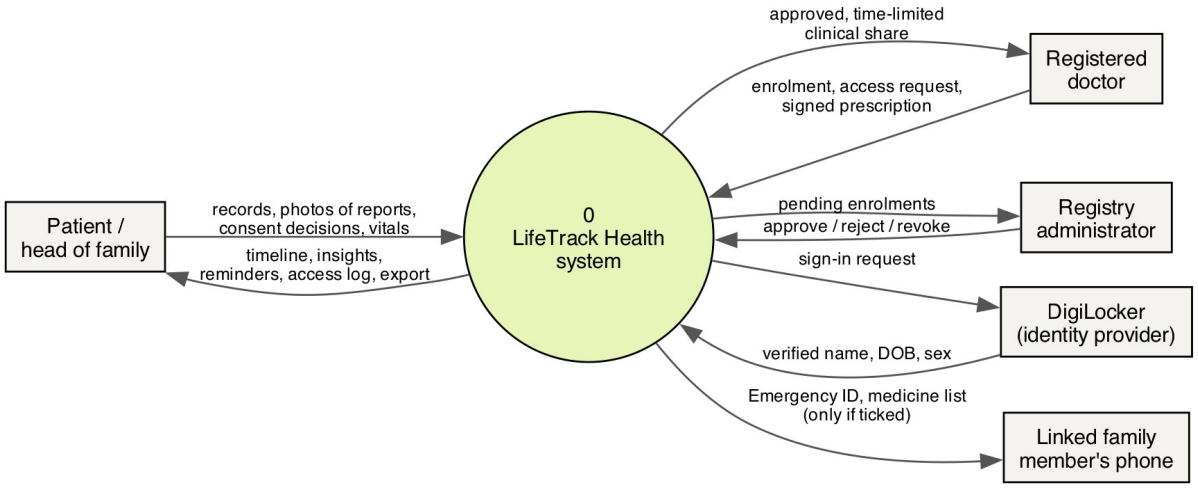


Figure 4.2: Level 0 data flow diagram

4.3 Patient app architecture

Inside the patient app (Figure 4.3), screens never touch storage or compare values with ranges themselves. They call AppServices, the single entry point, and render Insight objects. Any reader other than the patient (on-device insights, a share, the disabled cloud plane) must go through RecordAccess, which asks ConsentStore whether a stored grant allows the read and writes an audit entry for every read and every refusal. Below that, SealedStore encrypts every blob and FileByteStore writes it atomically (write to a temporary file, flush, rename).

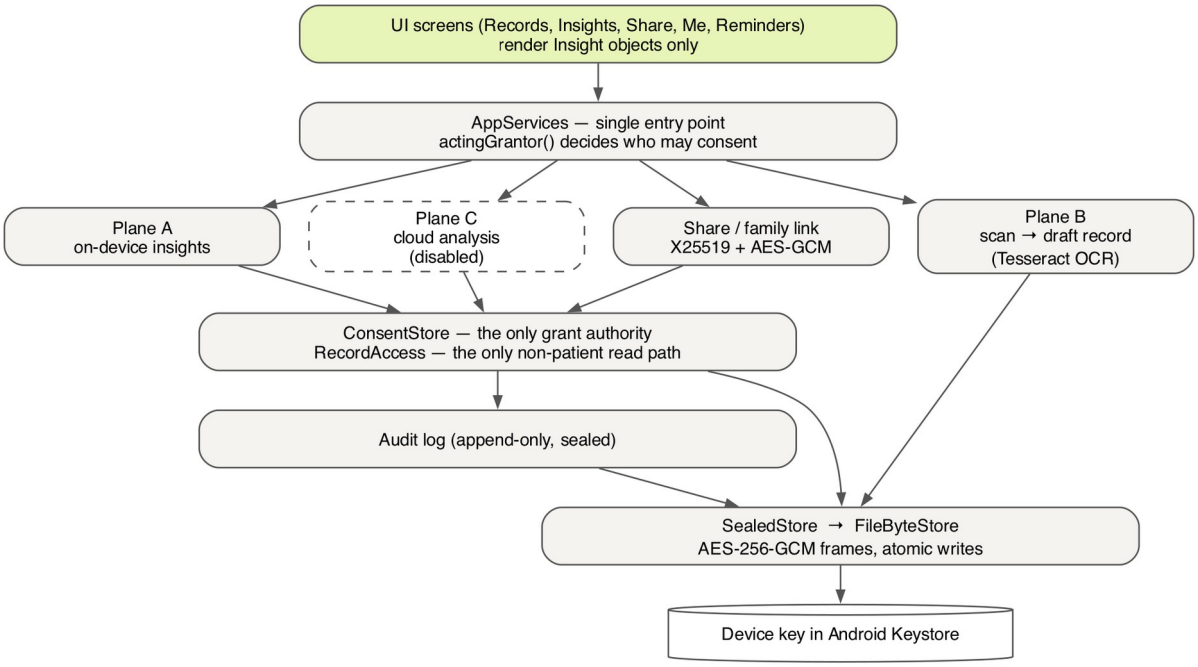


Figure 4.3: Layers of the patient app

The three planes

- **Plane A: on-device insights (Tier 1).** Fixed rules that state where a value sits relative to a published range, whether it moved meaningfully since the last report, and whether

two prescriptions share an ingredient. It reads records only under its own actor and a 24-hour grant the patient turns on, and it is forbidden from calling the clock directly so that its output is reproducible in tests.

- **Plane B: capture.** Turns a photographed or PDF page into a draft record using Tesseract OCR on the phone. Fields read with low confidence must be ticked or corrected before one final confirmation.
- **Plane C: cloud analysis (Tier 2).** Designed but switched off. It can only cross on a fresh human tap bound to the disclosure the patient read, with its own one-time grant. It stays dark until a certified service exists.

4.4 Consent model

A grant (consent token) records the patient, who granted it (the patient, or a parent or custodian for a dependent), the grantee, the purpose, the level and filter, the validity window, and whether it is revoked or spent. Authorisation always looks up the stored grant by id and re-checks every field; a token presented by a caller is never trusted on its own. Further rules:

- No grant exceeds 48 hours. A clinical share lasts one hour unless the patient chooses 4 h, 12 h, 1 day or 2 days; a doctor may ask for a time, and the patient may give less or change it later.
- A live grant cannot be silently re-issued with any difference; changing the time replaces the doctor’s copy with one that ends at the new time.
- No component under the AI planes may create, extend or exercise a grant it was not given.
- Proxy consent applies only to dependents: a parent for a child under 18 (sharing stops at 18), or a custodian for an adult who delegated in person. One function, actingGrantor(), makes that decision for every share.

Table 4.1: Access levels

Level	What the doctor sees
Summary (default)	Allergies, conditions, active medicines and the latest result for each test; the level aimed at preventing repeat tests
All my reports	Every report, prescription and scan, optionally filtered by date or test
Family illnesses	Illnesses in the family as the patient reports them, marked “patient-reported”
Home readings	Blood pressure and sugar readings from the last 30 days; off unless ticked

4.5 Data design

The main entities are Patient (profile, date of birth, sex, ABHA, role), Custody (who may consent for whom, and the family role), HealthRecord (kind: prescription, report, scan or note; source: typed by patient, smart capture, doctor-issued, restored, lab-signed; provenance badge; test results with the lab's printed range; medicines with status), ClinicalProfile (allergies, conditions, family illnesses), VitalReading, EmergencyCard (fixed, bounded fields), ConsentToken, and AuditEntry. Every record keeps where it came from, because a doctor will reuse a result instead of re-ordering the test only if it can be traced to a laboratory. Results show their age and never "valid" or "expired".

Each entity is serialised to JSON as `{schemaVersion, body}` and sealed into a frame `"LT" ‖ version ‖ nonce ‖ MAC ‖ ciphertext`. Version 2 frames bind the blob's name as associated data, so a blob copied under another name will not open. The index of blob names is itself sealed.

4.6 Security design

Clinical share

Figure 4.4 shows the complete share sequence. The doctor's device generates its own key pairs; the registry signs a certificate binding the doctor's registration number to those keys. The patient's phone verifies the certificate against the registry key pinned in the app and, when online, asks the registry that the doctor is still approved with the same keys. Only then does it show the doctor's name and registration number. The share is encrypted on the phone to the doctor's X25519 key; the audit entry is written before the bundle leaves; the relay deletes it after pickup or at expiry.

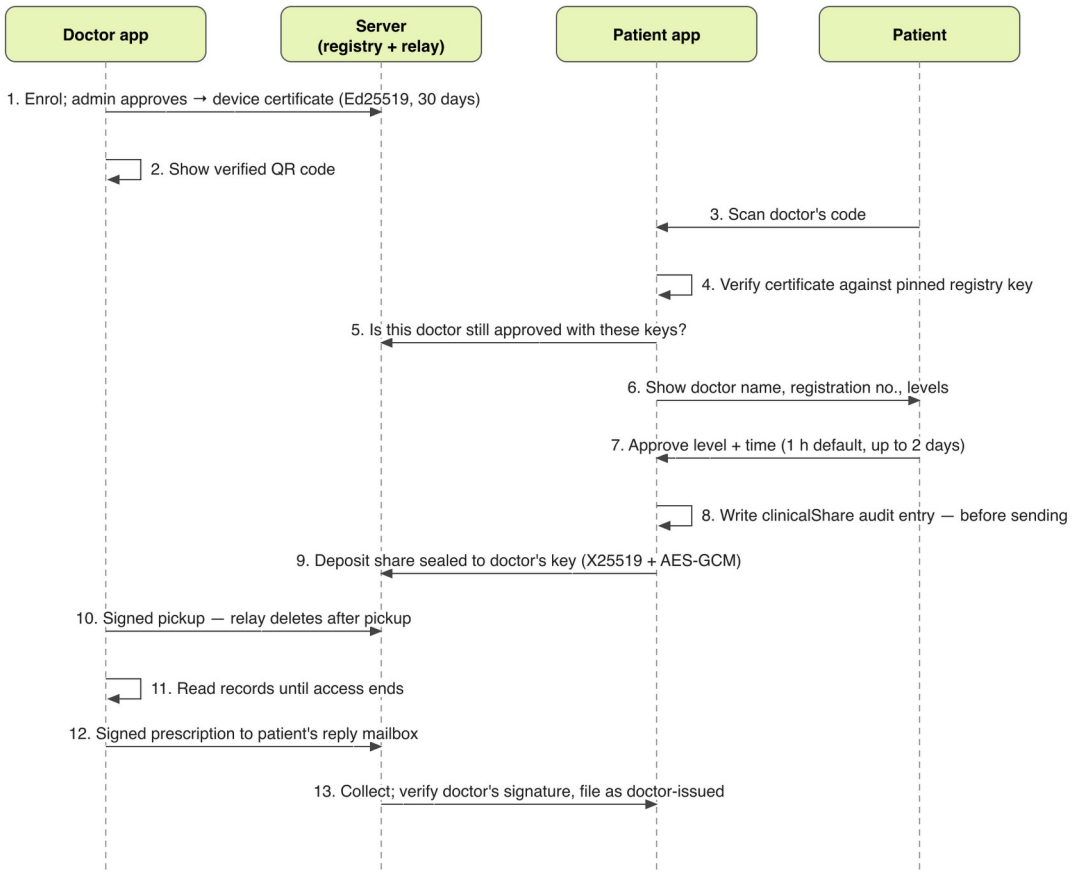


Figure 4.4: Clinical share and prescription sequence

Backup vault

User-chosen passwords are typically weak, and in an encrypted backup the password is the weakest link. LifeTrack therefore generates the password: 20 characters, about 100 bits of entropy, shown once with an account code and recovery codes. Figure 4.5 shows the key hierarchy. Argon2id stretches the password; HKDF derives a vault key and a login verifier. The vault key wraps a random data key, which encrypts the backup. The data key is also split into three Shamir shares, any two of which recover it. The server keeps only a hash of the verifier and rate-limits wrong proofs to ten per hour, a limit that survives restarts.

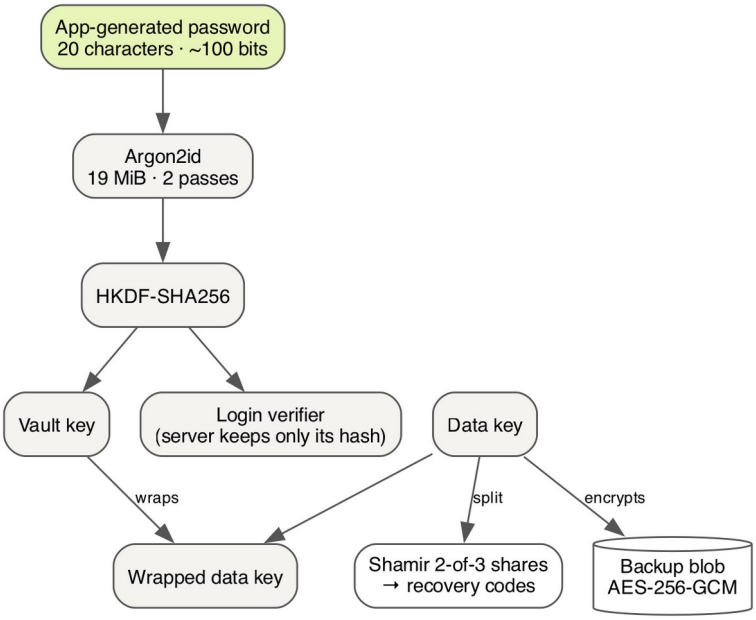


Figure 4.5: Backup key hierarchy

Other protections

- **App lock.** The app opens behind the phone's own lock (fingerprint, face or PIN). A phone without a screen lock is asked to set one and the records stay closed. The app re-locks after one minute away.
- **Screen privacy.** Screenshots and the recent-apps preview are blocked by default.
- **No plaintext on disk.** Camera files, picked PDFs and share-sheet files are deleted after use and swept at start-up.
- **Network discipline.** One file in the app is allowed to use the network; a test fails the build if any other file becomes socket-capable.
- **Sender proof.** A share that carries a LifeTrack ID also proves it came from the phone holding that ID's reply key, so a public ID lookup cannot be used to forge a share.
- **Server hardening.** Bound to 127.0.0.1 by default; DNS-rebinding and cross-origin pages refused; body size limits; per-caller rate limits; ID lookups throttled so the number space cannot be walked; enrolments awaiting review cannot be replaced; revocation follows keys.

4.7 LifeTrack ID

Each profile gets a random 12-digit LifeTrack ID shown as `XXXX XXXX XXXX`. The last digit is a Verhoeff check digit, which catches every single-digit error and every swap of neighbouring digits. IDs always start with 1, so they can never be confused with an Aadhaar

number. The server maps an ID only to a reply mailbox and key. It holds no name and no records, and only the token held on the phone can move or delete it.

4.8 Entity-relationship model

Figure 4.6 shows how the entities in Section 4.5 relate. Everything belongs to a Patient profile. A profile has many records, vital readings, grants and audit entries, and exactly one clinical profile and one Emergency ID. Custody links a grantor (a parent or custodian) to a dependent profile. Each grant has its own trail of audit entries, so every read made under a grant can be listed.

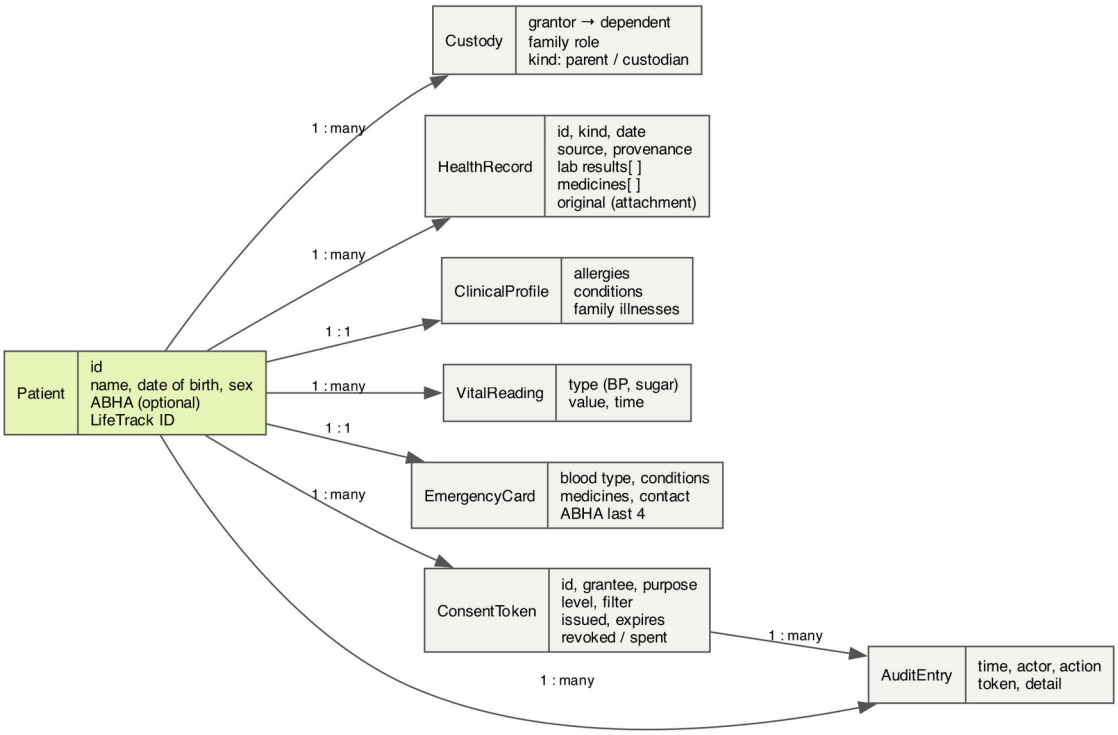


Figure 4.6: Entity-relationship diagram

Because the data lives on the phone as sealed JSON documents rather than in a relational database, the relationships are held as identifiers inside the documents. The audit log is append-only and is also sealed on disk.

4.9 Key algorithms

Verhoeff check digit

The twelfth digit of a LifeTrack ID is computed with the Verhoeff algorithm [16], which uses the multiplication table of the dihedral group D5, a permutation table and an inverse table. Unlike a simple sum-of-digits check, it detects every single-digit error and every transposition of two adjacent digits, the two most common mistakes when an ID is read out or typed. The

first digit is fixed at 1 so that no LifeTrack ID can be mistaken for, or validate as, an Aadhaar number.

Password derivation

The backup password is stretched with Argon2id [13] using the OWASP minimum parameters [14]: 19 MiB of memory (19,456 KiB), 2 passes and 1 lane, producing a 32-byte root key. At this cost the derivation is slow enough to drop frames, so it runs on a background isolate. HKDF-SHA256 [12] then derives two independent 32-byte keys from the root using different labels: the vault key (lifetrack/v1/vault-key), which never leaves the phone, and the login verifier (lifetrack/v1/login-verifier), which is sent to the server so that it can release the locked backup. Test builds use weaker parameters, and that code path throws an error in a release build.

Shamir secret sharing

The random data key that encrypts the backup is split with Shamir's scheme [15] over the finite field GF(2^8) into three shares, any two of which rebuild it; one share alone reveals nothing. Each share carries its index, the threshold and an identifier of the split, and is printed as a recovery code of the form LT1-index-threshold-splitId-hex. Combining fewer shares than the threshold, or shares from different splits, is refused instead of producing a wrong key.

Insight rules

Plane A compares the latest result for each test with a reference interval for the patient's age, sex and pregnancy status, checks it against a critical-value table, and looks for movement between consecutive results. A change counts as a trend only when it is larger than a fixed threshold for that test (Table 4.2). Each finding has one of four severities: a note, discuss at the next visit, contact a doctor today, or urgent care.

Table 4.2: Trend thresholds used by Plane A

Test	Unit	Minimum change	Worse direction
HbA1c	%	0.5	Up
Fasting glucose	mg/dL	15	Up
LDL cholesterol	mg/dL	20	Up
HDL cholesterol	mg/dL	8	Down
Haemoglobin	g/dL	1.0	Down
Creatinine	mg/dL	0.3	Up

Chapter 5: Implementation

5.1 Technology stack

Table 5.1: Technology stack

Layer	Technology	Reason
Patient app	Flutter (Dart), Kotlin for Android platform code	One code base, native performance, strong typing
Doctor app	Flutter (web and mobile)	Shares the protocol package and design language with the patient app
Server	Dart with shelf and shelf_router	Same language and protocol package as the apps; small and testable
Cryptography	cryptography package: AES-256-GCM, X25519, Ed25519, HKDF, HMAC-SHA256, Argon2id	Audited primitives; no home-made algorithms
Key storage	Android Keystore via flutter_secure_storage	Hardware-backed device key
OCR	Tesseract 4 (Tesseract4Android), English “best” model, checksum pinned	Fully offline; chosen over a cloud SDK that sends usage statistics
QR	flutter_zxing (scan), qr_flutter (draw)	Offline, no network permission
Other	local_auth, flutter_local_notifications, archive (encrypted zip), printing / pdf, share_plus	App lock, reminders, export, prescriptions

5.2 Code organisation and size

Table 5.2: Code size by part

Part	Folder	Source files	Lines of code
Patient app	app/lib	99	16,083
Doctor app (LifeTrack Clinic)	doctor/lib	16	3,292
Server	server/lib, server/bin	8	1,599
Shared protocol	packages/lifetrack_protocol/lib	8	713
Total application code		131	21,687
Tests (all parts)	*/test	58	7,097

The patient app is divided into modules under `app/lib/src/`: ai (planes A, B, C and the Insight contract), app (AppServices and wiring), audit, capture (OCR and image handling), consent, crypto, emergency, export, family, identity, model, reminders, security, share, storage, ui, vaccines and vault.

5.3 Patient app

The app has five tabs (Reminders, Records, Insights, Share and Me) and a family bar at the top for switching between profiles (Figure 5.1).

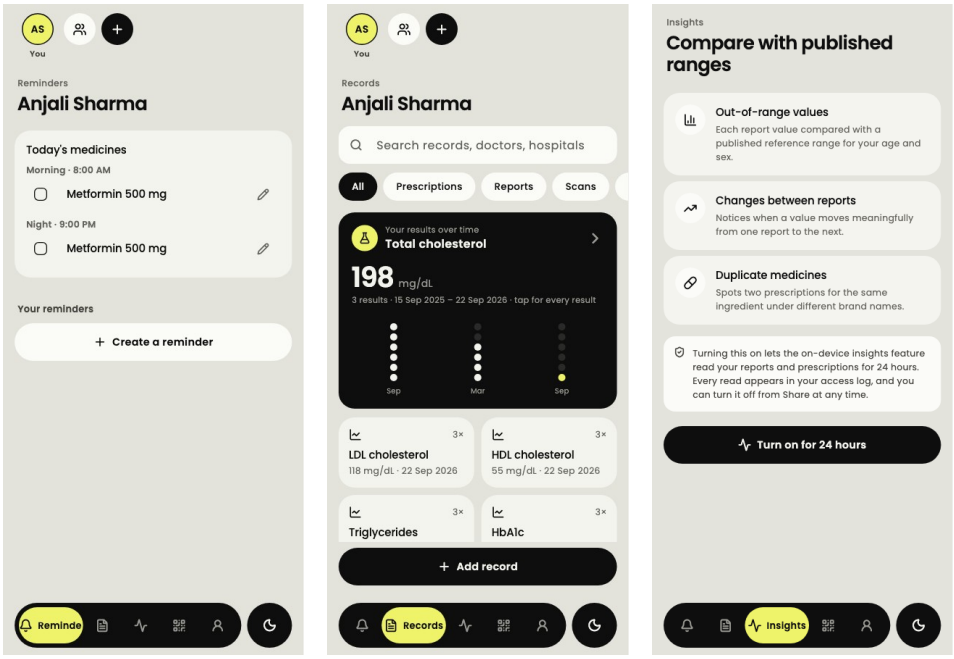


Figure 5.1: Patient app: Reminders, Records and Insights

- **Reminders** lists today's medicines from active prescriptions, each with a tick, and the patient's own reminders.
- **Records** shows a searchable timeline filtered by prescriptions, reports and scans, and a "results over time" card per test. Tapping a test opens every result with its laboratory, sample date and the lab's printed range.
- **Insights** explains what the on-device insights do and asks the patient to turn them on for 24 hours. Every read they make appears in the access log.

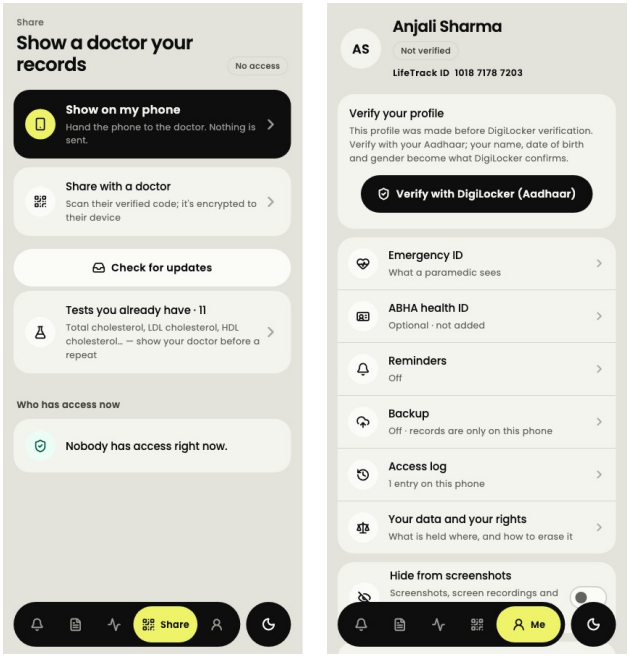


Figure 5.2: Patient app: Share and Me

- **Share** offers “Show on my phone” (no network), scanning a doctor’s code, “Tests you already have” to show a doctor before they order a repeat, and “Who has access now”, where any share can be shortened, extended or ended.
- **Me** holds the Emergency ID, ABHA (optional, “not verified”), reminders, backup, the access log, data rights (export and erasure), screenshot privacy, appearance and the family.

Scanning a report (Plane B)

A page arrives from the camera, the photo library, a PDF or another app’s share sheet. The image is straightened, resized and stripped of metadata, then passed to Tesseract [18] in the Android activity. A parser recognises test names, values, units and printed ranges. Any field read with confidence below 0.85, and every handwritten field, must be ticked or corrected before the patient confirms that the draft matches the report. The original image is kept, encrypted, with the record.

Insights (Plane A)

The engine takes records already obtained through RecordAccess and a clock, and returns Insight objects. A value is compared with a reference interval only when the units match and the interval applies to the patient’s age, sex and pregnancy status; otherwise it is reported as “not checked”, never as normal. Critical values change only the severity and suggested action (discuss soon, contact a doctor today, urgent care), never the wording of the finding. Every output is screened against a list of banned phrases covering diagnoses (“you have”) and forecasts (“risk of”, “may indicate”); a banned output is dropped, not rephrased.

Family

The phone owner is the head of the family. Each other profile has a role describing who the person is to the head. A child under 18 can be added only as the head’s son or daughter, with consent declared by the parent; sharing for that child stops at 18. An adult may delegate to a custodian in person, and can end it. Adults with their own phones can be linked. Each may share their Emergency ID and/or current medicine list with the other (both are off by default), sealed to the other phone’s key via the relay, with a safety code shown on both phones.

Emergency ID

The Emergency ID holds a fixed set of bounded fields (blood type, conditions, medicines, emergency contact, last four digits of ABHA). It is sealed like every other blob but can be

opened from the app's lock screen, and every such read is logged as an unauthenticated lock-screen read. Its QR code is plain text so any camera can read it without the internet.

5.4 Doctor app: LifeTrack Clinic

On first launch the doctor enrols with their name, registration number, council and specialty; the device generates its keys and the administrator approves the enrolment. The approved doctor's home screen shows their verified code for patients to scan, a form to request access by LifeTrack ID, and the shares that have arrived, each with a countdown (Figure 5.3).

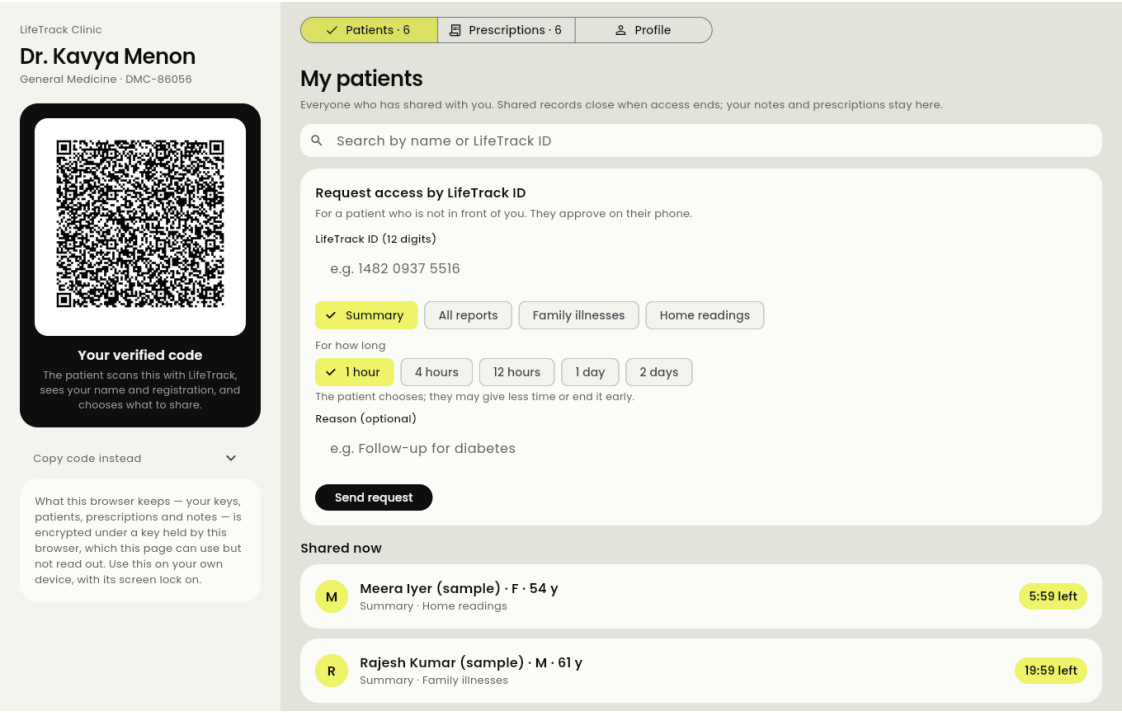


Figure 5.3: LifeTrack Clinic: verified code, request by ID and an arrived share

Opening a share shows the patient's allergies prominently, current and past medicines with their status, conditions and the latest tests with their source and age, and a link to the original report image. Beside it, the doctor writes a prescription with the medicine, active ingredient, morning-afternoon-night dose, timing relative to food, duration and a note, then signs and sends it (Figure 5.4). The prescription can also be printed with the registration number and a signature fingerprint. The doctor app keeps its own notes and a patient register, sealed in the browser.

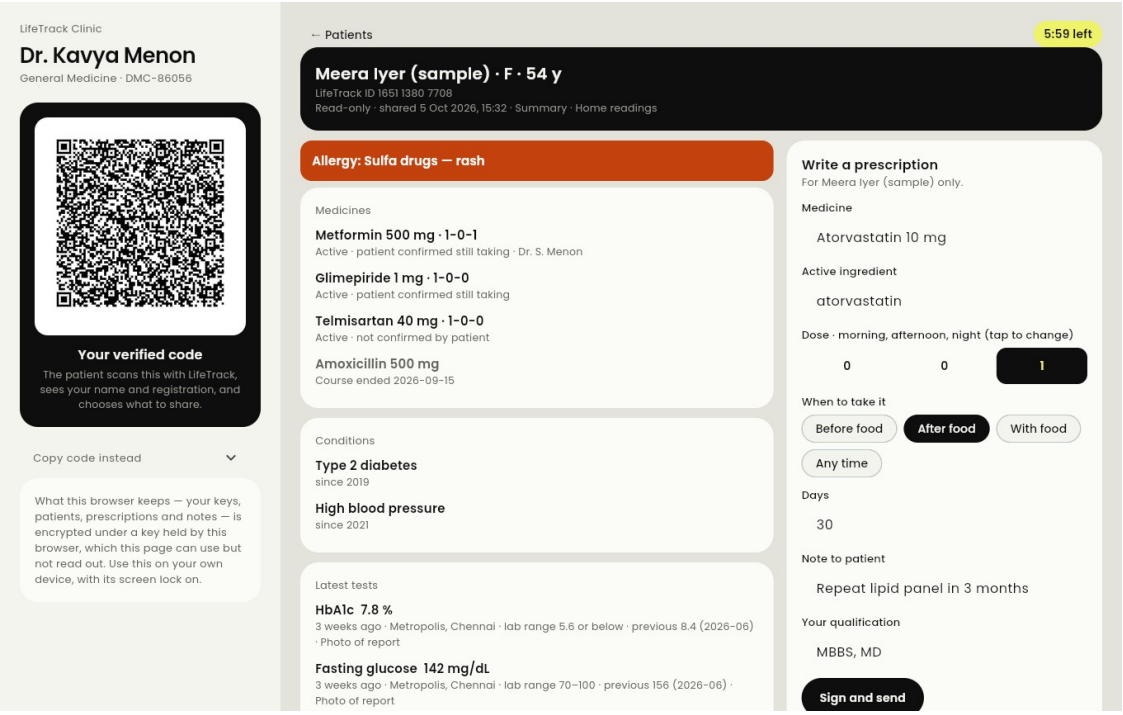


Figure 5.4: LifeTrack Clinic: a shared summary and a prescription being written

The app has three tabs: Patients, Prescriptions and Profile. The Profile tab (Figure 5.5) shows the licence details from the approved enrolment, which the doctor cannot edit, and when the certificate next renews. It also holds the practice details printed on paper prescriptions, the privacy settings (the app lock is always on, and hiding from screenshots is a switch), the appearance setting and a count of what this device keeps.

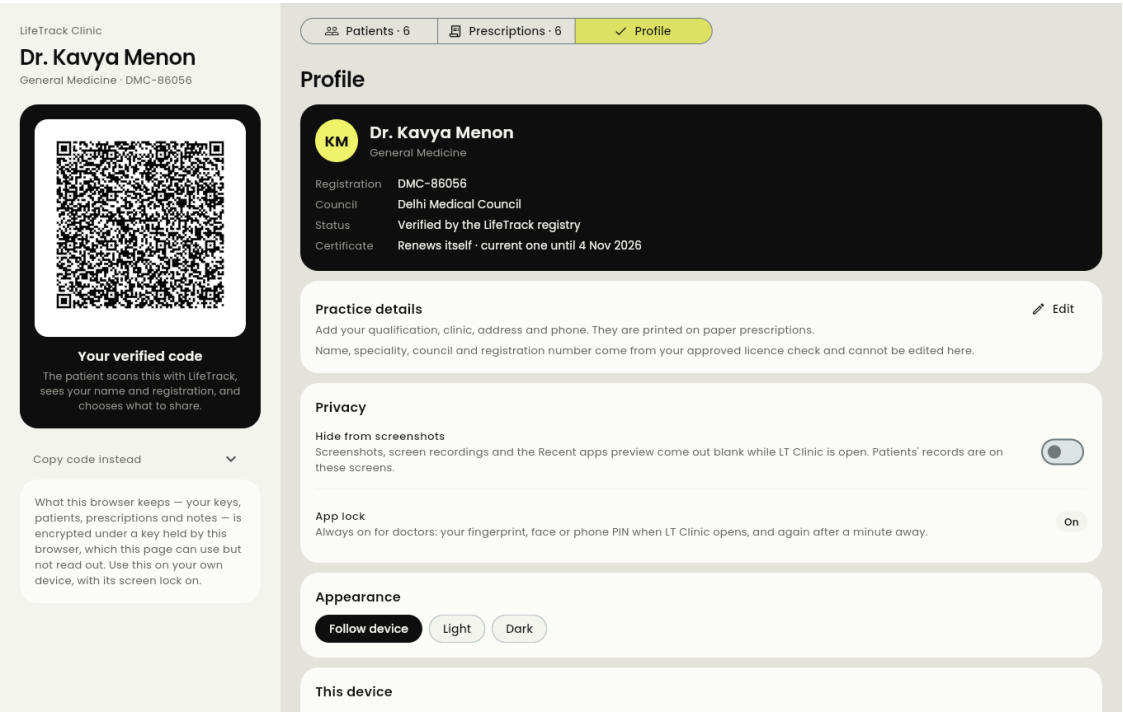


Figure 5.5: LifeTrack Clinic: the Profile tab

5.5 Server

The server is a single Dart process using shelf. Its routes are grouped under `/v1/`:

Table 5.3: Server routes

Route group	Purpose
/v1/health, /v1/registry/key	Health check; registry public key
/v1/doctors, /v1/admin/doctors/…	Enrolment, status lookup; administrator approve, reject, revoke (admin token required)
/v1/relay/<mailbox>	Deposit and signed pickup of sealed messages; deleted after pickup or at expiry; never beyond 48 hours
/v1/ids, /v1/ids/<id>	Register, move, delete and look up LifeTrack IDs (rate-limited)
/v1/vault/<account>/…	Salt, open, replace and delete backups with a password proof; ten wrong proofs per hour
/v1/verify/…	Identity check: start, callback, result released only with the hand-back code

By default the server binds only to 127.0.0.1. During development the phone reaches it over USB with `adb reverse tcp:8787 tcp:8787`. Data is kept in an owner-only folder: the registry signing key, the admin token, doctor enrolments, the ID map and the backup files.

5.6 Development practices

- **Version control.** Git with a private GitHub repository; every change is a reviewed commit.
- **Versioning rule.** Every update raises the version (a feature raises the middle number, a fix the last) and adds a line to the changelog in the same commit. At the time of writing the patient app is at 1.19.0 and the doctor app at 1.8.0 (Appendix A).
- **Security reviews.** Two dedicated hardening releases (patient 1.13.0 and 1.18.1/1.18.2, doctor 1.4.1/1.4.2) fixed issues found in review, such as binding the identity result to a hand-back code and proving the sender of a share.
- **Release build.** A script builds an obfuscated APK; the symbol map is kept on the development machine.

5.7 Selected implementation details

This section shows four short excerpts from the patient app’s source code. They are the places where the main rules of Chapter 4 are enforced. Listings are shortened only where marked with “...”.

Consent check

Every read by anyone other than the patient ends in ConsentStore.authorize (Listing 5.1). The caller passes only the grant id; the method looks up the stored grant and checks every field, in a fixed order, against the request. Each failure throws a typed denial, which RecordAccess writes to the audit log as a refusal. A one-time grant, such as the one Plane C would use, is marked spent on its first use.

```
ConsentToken authorize({required String tokenId, required String actorId,
 required String patientId, required DataClass dataClass,
 required GrantPurpose purpose, required DateTime at}) {
  final t = _issued[tokenId];
  if (t == null) throw ConsentDenied(DenialReason.unknown, tokenId);
  if (_revoked.contains(tokenId)) throw ConsentDenied(DenialReason.revoked, tokenId);
  if (_spent.contains(tokenId)) throw ConsentDenied(DenialReason.spent, tokenId);
  if (at.isBefore(t.issuedAt)) throw ConsentDenied(DenialReason.notYetValid, tokenId);
  if (!at.isBefore(t.expiresAt)) throw ConsentDenied(DenialReason.expired, tokenId);
  if (t.patientId != patientId) throw ConsentDenied(DenialReason.wrongPatient, tokenId);
  if (t.grantedToActorId != actorId) throw ConsentDenied(DenialReason.wrongActor, tokenId);
  if (t.purpose != purpose) throw ConsentDenied(DenialReason.wrongPurpose, tokenId);
  if (!t.covers(dataClass)) throw ConsentDenied(DenialReason.outOfScope, tokenId);
  if (t.type == GrantType.oneTime) _spent.add(tokenId);
  return t;
}
```

Listing 5.1: ConsentStore.authorize (lib/src/consent/consent_store.dart)

Sealed storage

All records, grants and audit entries pass through SealedStore (Listing 5.2). Each blob is encrypted under the device key and bound to its own name, so a blob copied to another name fails to open. A blob in the older version-1 frame is upgraded to version 2 the first time it is read.

```
Future<Object?> read(String name) async {
  final frame = await bytes.read(name);
  if (frame == null) return null;
  final body = await VaultCodec.open(frame, key, name: name);
  if (VaultCodec.versionOf(frame) == 1) await write(name, body);
  return body;
}

Future<void> write(String name, Object? body) async =>
    bytes.write(name, await VaultCodec.seal(body, key, name: name));
```

Listing 5.2: SealedStore read and write (lib/src/storage/sealed_store.dart)

Audit before egress

Listing 5.3 is the end of the clinical-share flow. The bundle is sealed to the doctor’s key first, then the audit entry is written, and only then is the envelope handed to the relay. If the network call fails, the attempt is still on record.

```
final envelope = await ShareEnvelope.seal(bundle: bundle,
    recipientKey: doctor.encryptionKey, keyId: keyId,
    grantId: grant.id, expiresAt: grant.expiresAt);
// Audited before it leaves: a failed send is still recorded.
await audit.append(AuditEntry(at: at, actorId: grant.grantorId,
    patientId: patientId, action: AuditAction.clinicalShare, tokenId: grant.id,
    detail: 'Sent to ${doctor.name} (${doctor.registrationNo}), encrypted to their
device'));
await relay.deposit(mailbox: mailbox, grantId: grant.id,
    envelope: envelope, expiresAt: grant.expiresAt);
```

Listing 5.3: Writing the audit entry before the share is sent (lib/src/share/clinical_share_service.dart)

Banned-phrase filter

Every Plane A output is checked against a list of banned phrases covering diagnoses, forecasts and treatment advice (Listing 5.4). An insight that contains one is dropped, not reworded, so the filter can only remove output and never invent new wording.

```
const bannedPhrases = <String>[
  // assertions
  'diagnos', 'you have', 'you are', 'disease', 'is normal', 'healthy', ...
  // forecasts and hedges
  'risk of', 'likely', 'may indicate', 'suggests', 'progression to', 'predict', ...
  // treatment advice
  'stop taking', 'increase your dose', 'reduce your dose', 'skip',
];

String? firstBannedPhrase(String text) {
  final t = text.toLowerCase();
  for (final p in bannedPhrases) {
    if (t.contains(p)) return p;
  }
  return null;
}
```

Listing 5.4: Banned-phrase list and check, shortened (lib/src/ai/plane_a/banned_phrases.dart)

Vaccination schedule

The child vaccination schedule follows India’s Universal Immunisation Programme as published by the Ministry of Health and Family Welfare [7]: 26 doses from birth (BCG, OPV 0, hepatitis B) to 16 years. Each dose has an age at which it falls due, and the app computes due dates from the child’s date of birth. The screen advises following the child’s doctor where a private schedule adds vaccines.

Chapter 6: Testing and Results

6.1 Testing strategy

Testing followed one principle from the project proposal: a claim is accepted only if a test can prove it. Five kinds of tests were developed:

- **Unit tests** for pure logic: reference ranges, trend thresholds, Verhoeff digits, Shamir splitting, frame encoding, grant checks.
- **Widget tests** that drive real screens: adding records, sharing, back-button behaviour, dark mode.
- **Invariant tests** that read the source code itself and fail the build if a rule is broken. For example, the build fails if any file other than the one network file can open a socket, if the release manifest gains the INTERNET permission without review, if code under `lib/src/ai/` calls `ConsentStore.issue` or `DateTime.now()`, or if any UI string claims that LifeTrack or the server "cannot read" the patient's data. These tests strip comments first, so a comment cannot hide or fake a violation.
- **Contrast tests** that walk every visible text on every main screen, in light and dark mode, to the solid colour behind it and check WCAG AA contrast.
- **Integration tests** that run the real code of both apps against the real server handler: scan, share, confirmed sender, prescription back, request by ID, decline, time change and revocation.

6.2 Results

All tests were run on 5 October 2026 against the current version (patient app 1.19.0, doctor app 1.8.0). Static analysis (`flutter analyze` / `dart analyze`) reported no issues in any package.

Table 6.1: Test results by part

Part	Static analysis	Tests passed	Failed
Patient app (app/)	No issues	285	0
Doctor app (doctor/)	No issues	18	0
Server (server/)	No issues	37	0
Shared protocol	No issues	8	0
Both apps + server (integration/)	No issues	6	0
Total		354	0

One additional test in the patient app is a tool that renders screens to images for visual review; it runs only when asked to save screenshots and is not counted.

6.3 Selected test cases

Table 6.2: Selected test cases

#	Test case	Expected result	Result
1	A pending (unapproved) doctor tries to show a code	No scannable code is produced	Pass
2	Patient scans, shares; doctor opens; doctor prescribes; patient collects	Share opens only with the doctor's key; prescription verified as signed and filed as doctor-issued	Pass
3	Doctor requests by LifeTrack ID; patient approves once, declines once	Approval produces a share; decline is logged under Refused and the doctor is told	Pass
4	A doctor is revoked after enrolment	Patient app refuses the doctor; doctor app stops showing as approved	Pass
5	Doctor asks for 12 h; patient gives 4 h, later changes it	Doctor sees each new end time; old copy replaced	Pass
6	Share made without choosing a time	Access lasts exactly 1 hour	Pass
7	A doctor tries to pick up another doctor's mailbox	Refused	Pass
8	A signed pickup is replayed	Refused	Pass
9	Ten wrong backup passwords	Account locked for an hour; server keeps only a hash	Pass
10	An ID is mistyped by one digit or two neighbours swapped	Rejected by the Verhoeff check before any lookup	Pass
11	A sealed blob is copied under another name	Fails to open (name bound as associated data)	Pass
12	A unit on a report does not match the reference range's unit	Shown as "not checked", never as out of range or normal	Pass
13	An insight's wording contains a banned diagnostic or forecast phrase	Insight dropped	Pass
14	A DigiLocker sign-in link is forwarded to someone else	Result released only with the hand-back code on the finishing page; five wrong codes end the check	Pass
15	Any visible text in light or dark mode	Meets WCAG AA contrast	Pass

6.4 Manual testing on a device

The release APK was installed on an Android phone (Samsung Galaxy S25) connected to the local server over USB, and the share, request-by-ID, prescription and backup flows were tested manually. A medicine reminder fired on the device with medicine names hidden on the lock screen, as designed. OCR has so far been tested mainly on synthetic report images; evaluation on a wider set of real photographed reports is future work.

6.5 Example of an invariant test

Listing 6.1 is taken from the invariant test that guards the AI planes. It loads the source of every file under lib/src/ai, strips the comments (codeOf), and fails if any of them calls ConsentStore.issue, imports the grant code, or reads the wall-clock time. If a future change gave an agent a way to grant itself access, the build would fail before the change could be released.

```
group('agents (lib/src/ai) never', () {
  final ai = [for (final f in lib) if (rel(f).startsWith('lib/src/ai/')) f];

  test('issue a grant (CLAUDE.md #5)', () {
    for (final f in ai) {
      final code = codeOf(f);
      expect(code.contains('.issue('), isFalse, reason: rel(f));
      expect(code.contains('grants.dart'), isFalse, reason: rel(f));
    }
  });

  test('read wall-clock time - the clock is always a parameter', () {
    for (final f in ai) {
      expect(codeOf(f).contains('DateTime.now()'), isFalse, reason: rel(f));
    }
  });
});
```

Listing 6.1: Part of plane_boundary_test.dart (app/test/invariants)

Chapter 7: Results and Discussion

7.1 Objectives achieved

Table 7.1: Objectives and outcomes

Objective	Outcome
1. Patient-owned, encrypted record for the family	Achieved. All data sealed with AES-256-GCM under a hardware-backed key; family with head-of-family roles
2. Explicit, time-limited, revocable clinician access	Achieved. Four levels; 1 h default up to 2 days; change or end at any time; every share audited before it leaves
3. No server can read shares or backups	Achieved by design and tested: shares sealed to the doctor's key; backups to a key from an app-generated password
4. Safe on-device insights	Achieved for ranges, movement and duplicate medicines; non-diagnostic wording enforced by tests
5. On-device reading of reports	Achieved on Android with Tesseract; English only; accuracy on real photos still to be measured
6. Verified doctor app with signed prescriptions	Achieved: registry certificates, status checks, signed and printable prescriptions
7. Verifiable privacy promises	Achieved: invariant tests in the build; 354 tests passing

7.2 Changes from the original proposal

The proposal planned a mobile app, a separate web analytics dashboard and a clinician portal, with a FHIR R4 data model and a synthetic-dataset approach to verifying analytics. Three changes were made to the proposal during development:

- **Analytics moved onto the phone.** A web dashboard would need a server that can read records, which contradicts the project's central promise. The analytics were therefore built into the patient app as on-device insights.
- **The clinician portal became LifeTrack Clinic**, a full doctor app with verified identity, request by ID and signed prescriptions, because safe sharing turned out to be the most useful feature for reducing repeat tests.
- **FHIR alignment was reduced to provenance.** Every record keeps its source and provenance so that a later FHIR export is possible, but FHIR resources and export were not implemented in this phase.

The proposal's "ground-truth" principle was retained and extended. Instead of one synthetic dataset, every rule the system promises is written as a test with a known expected result.

7.3 Discussion

The principal lesson of the project is that privacy must be part of the structure of the code from the outset; it cannot be added later as a feature. Making ConsentStore the only grant authority and RecordAccess the only non-patient read path meant that sharing, insights and the cloud plane all inherited the same checks and the same audit trail. Expressing the rules as source-scanning tests turned code review questions (“does anything else open a socket?”) into automatic answers.

A second lesson is that accurate wording is itself a requirement. Early screens said the doctor “keeps no copy” and that the server “cannot read” the data. Both claims were removed: a doctor is legally required to keep their consultation record, and an offline password-guessing attack on a human-chosen password would make “cannot read” untrue. The app now generates the password and the UI makes only claims the code can back, enforced by a test.

Finally, the work on provenance showed that a doctor will reuse a result instead of ordering it again only if they trust where it came from. Access control alone is not enough; the doctor needs the laboratory, the sample date, the printed range and the original image. These are shown on every result.

7.4 Comparison with existing solutions

Table 7.2 compares LifeTrack Health with the two most common existing categories from Chapter 2. The entries for existing categories describe typical systems in each category, not any single product.

Table 7.2: Comparison with existing categories

Feature	Hospital EMR	Document locker	LifeTrack Health
Patient holds the complete record	No	Documents only	Yes
Works offline	Inside the hospital	No	Yes
Results compared with ranges	Within one hospital	No	Yes, on the device
Time-limited sharing with one doctor	No	Link or consent artefact	Yes, 1 hour to 2 days
Doctor verified before sharing	Staff only	Varies	Yes, certificate
Server can read shared data	Yes	Yes	No (encrypted)
Family profiles, proxy consent	No	Limited	Yes
Privacy rules tested automatically	Not published	Not published	Yes (invariant tests)

Chapter 8: Conclusion and Future Scope

8.1 Conclusion

LifeTrack Health shows that a patient can hold a complete, useful health record on their own phone and share exactly the right part of it with a verified doctor for a limited time, while the server that carries the data can never read it. The prototype covers the full loop (capture, understanding, sharing, prescribing back, reminders, emergencies, family, backup and export) across a patient app, a doctor app and a server, and every privacy and safety promise it makes is checked by an automated test. All 354 tests pass and static analysis is clean.

8.2 Limitations

- Only synthetic patients and doctors have been used; no real patient data has been processed.
- The identity check uses a practice stand-in for DigiLocker until partner registration; the optional mode that requires Aadhaar-based verification for every profile must be reviewed by legal counsel in light of *Puttaswamy* (2018) and the 2019 Aadhaar amendment before any pilot.
- ABHA numbers are typed in and shown as “not verified”; ABDM integration is not yet built.
- OCR is English-only, Android-only, and not yet measured on a large set of real photographed reports.
- The critical-value table follows commonly published limits and must be replaced by a partner laboratory’s published list before a pilot.
- The server has run locally (over USB) and has not been deployed on the public internet.

8.3 Future scope

1. ABDM / ABHA integration: verify ABHA, link records as a health information user, and map ABDM consent artefacts as a separate long-lived type.
2. DigiLocker partner registration to replace the practice stand-in.
3. FHIR R4 export of records with Provenance resources.
4. Hosted deployment of the server behind a reverse proxy, with push notifications instead of polling.
5. iOS support, including on-device OCR with Apple Vision.

6. OCR accuracy study on real photographed reports from Indian laboratories.
7. Relative-contributed family history, with that relative's own consent per request.
8. Plane C (Tier 2) cloud analysis, only with a certified service and the human-tap safeguards already designed.
9. Pilot with a partner clinic and laboratory after legal and clinical review.

References

[1] Government of India. *The Digital Personal Data Protection Act, 2023* (Act No. 22 of 2023). Ministry of Law and Justice, 2023.

[2] Government of India. *The Aadhaar (Targeted Delivery of Financial and Other Subsidies, Benefits and Services) Act, 2016*, Section 29; and the Aadhaar and Other Laws (Amendment) Act, 2019.

[3] Supreme Court of India. *Justice K.S. Puttaswamy (Retd.) v. Union of India*, (2019) 1 SCC 1 (Aadhaar judgment, 26 September 2018).

[4] Ministry of Electronics and Information Technology. *Information Technology (Reasonable Security Practices and Procedures and Sensitive Personal Data or Information) Rules, 2011*.

[5] Board of Governors, Medical Council of India. *Telemedicine Practice Guidelines*, March 2020.

[6] National Health Authority. *Ayushman Bharat Digital Mission (ABDM)*: ABHA and Health Information Exchange & Consent Manager specifications. https://abdm.gov.in

[7] Ministry of Health and Family Welfare. *Universal Immunization Programme*: National Immunization Schedule.

[8] HL7 International. *FHIR Release 4 (R4)*. https://hl7.org/fhir/R4/

[9] M. Dworkin. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST SP 800-38D, 2007.

[10] A. Langley, M. Hamburg, S. Turner. *Elliptic Curves for Security*. RFC 7748, IETF, 2016.

[11] S. Josefsson, I. Liusvaara. *Edwards-Curve Digital Signature Algorithm (EdDSA)*. RFC 8032, IETF, 2017.

[12] H. Krawczyk, P. Eronen. *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. RFC 5869, IETF, 2010.

[13] A. Biryukov, D. Dinu, D. Khovratovich, S. Josefsson. *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*. RFC 9106, IETF, 2021.

[14] OWASP Foundation. *Password Storage Cheat Sheet* (Argon2id parameters). https://cheatsheetseries.owasp.org

[15] A. Shamir. “How to Share a Secret.” *Communications of the ACM*, 22(11), 612–613, 1979.

[16] J. Verhoeff. *Error Detecting Decimal Codes*. Mathematical Centre Tracts 29, Mathematisch Centrum, Amsterdam, 1969.

[17] M. Kleppmann, A. Wiggins, P. van Hardenberg, M. McGranaghan. “Local-First Software: You Own Your Data, in Spite of the Cloud.” *Proc. ACM Onward! 2019*.

[18] R. Smith. “An Overview of the Tesseract OCR Engine.” *Proc. ICDAR 2007*, IEEE.

[19] W3C. *Web Content Accessibility Guidelines (WCAG) 2.1*, 2018.

[20] Google. *Flutter documentation* and *Dart language documentation*. https://docs.flutter.dev, https://dart.dev

[21] IMDRF SaMD Working Group. *Software as a Medical Device: Possible Framework for Risk Categorization and Corresponding Considerations*, 2014.

Appendix A: Version History

Selected releases from the project changelog. A feature raises the middle number, a fix the last.

Table A.1: Version history

Version	Date	Highlights
Patient 1.0.0	29 Sep 2026	App scaffold, consent core, sealed storage, backup vault, Emergency ID
Patient 1.1–1.6	30 Sep – 1 Oct	Report capture with originals, family profiles, doctor sharing, edit and delete records
Patient 1.7–1.12	1 Oct	Medicine reminders, vaccinations, PDF import, “Tests you already have”, home readings
Patient 1.13.0	1 Oct	Security hardening
Patient 1.14–1.16	1 Oct	LifeTrack ID, request by ID, family roles, dark mode
Patient 1.17.0	2 Oct	On-device OCR (Tesseract) with confirm step
Patient 1.18.1–1.18.3	3 Oct	Security review fixes: hand-back code, proven family links, sender proof, back button
Patient 1.19.0	3 Oct	Choose and change how long a doctor can see the records
Doctor 1.0.0	30 Sep	Enrolment, verified code, view shares
Doctor 1.1–1.4	1 Oct	Prescriptions, printable PDF, request by ID, dark mode
Doctor 1.4.1–1.6.1	3 Oct	Security fixes, patient register, notes, “LifeTrack Clinic” branding, back button
Doctor 1.7.0–1.8.0	3 Oct	Ask for a time; Profile tab and screenshot setting

Appendix B: Running the System

Start the server

```
cd server && dart run bin/server.dart --practice-digilocker
```

The server prints its address (http://127.0.0.1:8787), the registry public key to pin in the apps, and where the admin token is kept.

Connect an Android phone

Enable USB debugging on the phone, connect it, and run `adb reverse tcp:8787 tcp:8787`. The phone's 127.0.0.1:8787 then reaches the server on the computer.

Build and install the patient app

`scripts/build_apk.sh` builds an obfuscated release APK with the server address and registry key; install it with `adb install`.

Run the doctor app

`cd doctor && flutter run -d chrome` (or on a phone). Enrol, then approve the enrolment with the admin token.

Run all tests

`flutter analyze && flutter test` in app/, doctor/ and integration/; `dart analyze && dart test` in server/ and packages/lifetrack_protocol/.